virtua

Pre-Staking

SMART CONTRACT AUDIT REPORT

IMMUNEBYTES

07 November 2023

Introduction **3**

About Virtua 3

About ImmuneBytes 3

Documentation Details **4**

Audit Goals **4**

Audit Process & Methodology **5**

Audit Details **6**

Security Level References **7**

Severity Status References **8**

Severity Status **8**

Finding **9**

Critical Severity Issues **10**

High Severity Issues **10**

Medium severity issues **10**

Low severity issues **10**

Informational **11**

Concluding Remarks **12**

Disclaimer **12**

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About Virtua

Virtua is a sustainable metaverse platform that provides immersive social, gaming, digital collectible and creative experiences, through NFTs and blockchain technology.

Visit https://virtua.com/ to know more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting smart contract audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 205+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The team has provided the following doc for audit:

1. TVK to VANRY Token Swap v2

Audit Goals

The focus of the audit was to verify that the smart contract system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within each contract and within the system of contracts.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established smart contract best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include

    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

ImmuneBytes team has performed thorough testing of the project, starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is a structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors which included -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

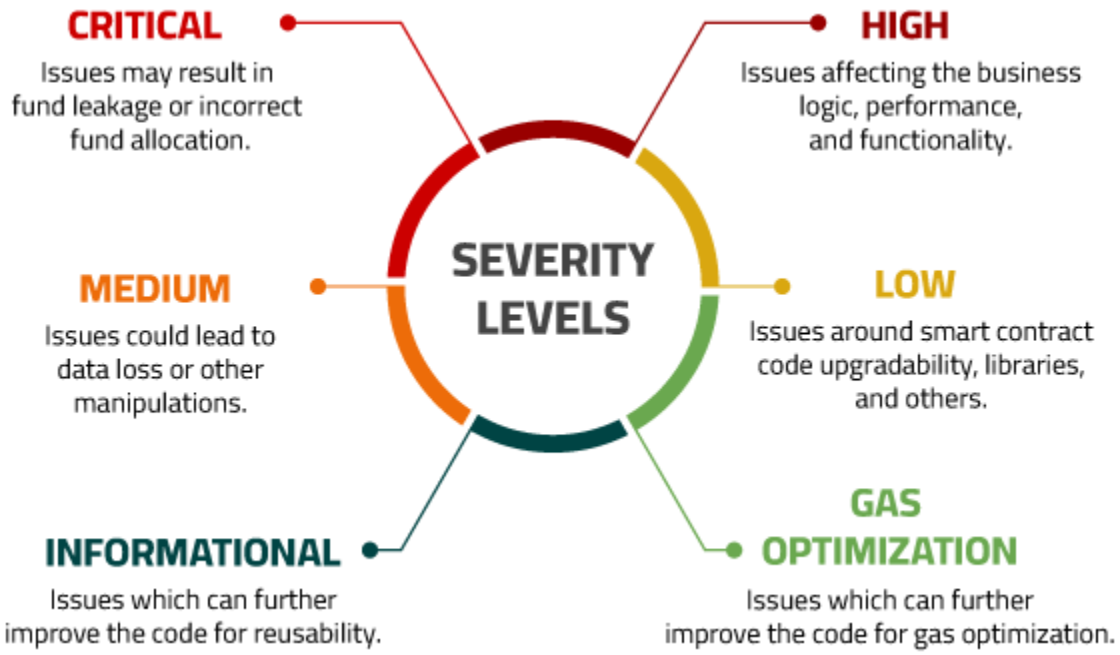
Audit Details

Project Name	Virtua
Platform	EVM
Contract	PreStakingContract.sol
Languages	Solidity
GitHub Link	https://github.com/TerraVirtuaCo/virtua-token/tree/lock-swap
Commit	c1cb434ef7980332a656a494c032d0693da94d3c
Platforms & Tools	Remix IDE, Truffle, VScode, Contract Library, Slither, SmartCheck, Fuzz

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:



This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Severity Status References

Severity Status

Issues	Critical	High	Medium	Low	Informational	Gas
Open	-	-	-	-	-	-
Fixed	-	-	-	2	1	-
Redacted	-	-	-	-	-	-
Acknowledged	-	-	-	-	-	-

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Finding

#	Findings	Risk	Status
1.	PreStakingContract: Redundant function can be removed.	Low	Fixed
2.	PreStakingContract: Absence of Zero Address Validation	Low	Fixed
3.	PreStakingContract: State Variable Could Be Marked as Immutable	Informational	Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

No issues were found.

High Severity Issues

No issues were found.

Medium severity issues

No issues were found.

Low severity issues

Issue	PreStakingContract: Redundant function can be removed.
Status	Fixed
Description	The use of balanceOf() function can be avoided by simply making the **stakedBalances** mapping public. It will directly let users check their staked balances whenever required. It also allows us to remove the balanceOf function and helps us reduce the contract size which eventually reduces the deployment cost for the contract.
Developer Comment	We are keeping this function intact due to some requirements at dApp level.
Auditor Comment	Acnkowledged

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Issue	PreStakingContract: Absence of Zero Address Validation
Status	Fixed
Description	The contract constructor fails to include any zero address validation check for the token address being passed during deployment. Including adequate validation is considered a better practice in solidity specifically for those state variables which aren't supposed to be updated later in the contract.
Recommendations	A **require** statement should be included in such functions to ensure no zero address is passed in the arguments.
Developer Comment	We have removed the token address from the constructor and marked it as constant at contract level. So this issue is no more relevant and can be considered as fixed.
Auditor Comment	Acknowledged

Informational

Issue	PreStakingContract: State Variable Could Be Marked as Immutable
Status	Fixed
Description	Upon a thorough examination of the contract, it was noted that the state variable **token** of type **IERC20** is currently set as a public state variable. However, its value is only set during the contract's deployment via the constructor and there are no subsequent functions within the contract that modify this value. Having a state variable without any updating mechanism and which remains unchanged post-deployment can introduce unnecessary gas costs during contract interactions due to its mutable nature.
Recommendations	To optimize the contract and ensure clarity regarding the invariability of the **token** state variable mark the **token state variable as Immutable**.
Developer Comment	we have marked it as constant at contract level as we know the value of the token even before the deployment of the token hence this issue is no more relevant and can be considered as fixed.
Auditor Comment	Acknowledged

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Concluding Remarks

While conducting the audits of Virtua PreStaking Contract, it was observed that the contracts contain only Low severity issues. Our auditors suggest that the developers should resolve issues on priority before moving up with production. The recommendations given will improve the operations of the smart contract.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the Virtua platform or its product, nor is this audit investment advice.

Notes:

- Please make sure contracts deployed on the mainnet are the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES

AUDITS